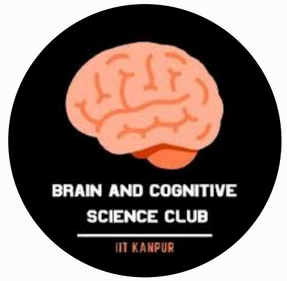
Y25 WINTER PROJECT



JUDGE IT WELL

End-Eval Presentation

SPECIAL THANKS TO Poonam Gupta
Aditya Goel
Shambhavi Singh

Project overview

Judge It Well is a project that fine-tunes a general-purpose LLM into a legal expert, enabling accurate interpretation, analysis, and reasoning over complex legal documents.

Problem Statement

Artificial Intelligence has advanced rapidly, yet legal text remains one of the most challenging domains for AI. Legal language is:

- Highly technical
- Context-dependent
- Structured but linguistically unique

General LLMs cannot fully understand or reason through legal documents without domain-specific training. Judge It Well bridges this gap by fine-tuning LLMs on legal data, enabling them to process statutes, contracts, judgments, and legal queries more effectively.

Solution

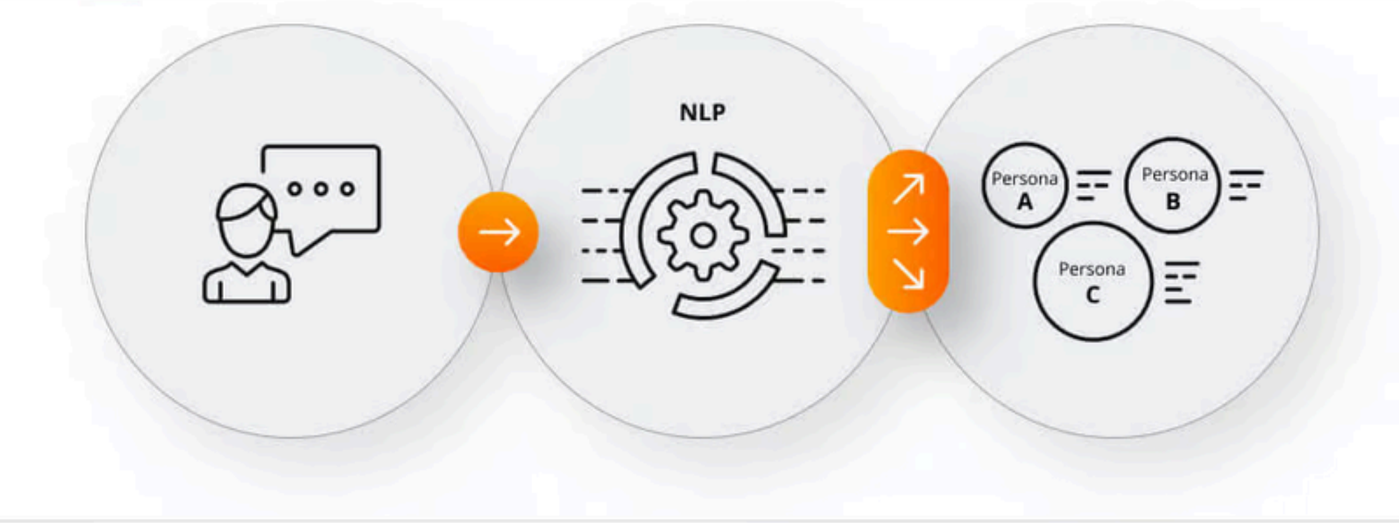
Judge It Well fine-tunes a large language model to act as a legal expert, enabling it to analyze complex legal documents, perform legal reasoning, support tasks like summarization and Q&A, and evaluate performance using legal benchmarks.

Table of Content

Judge it Well

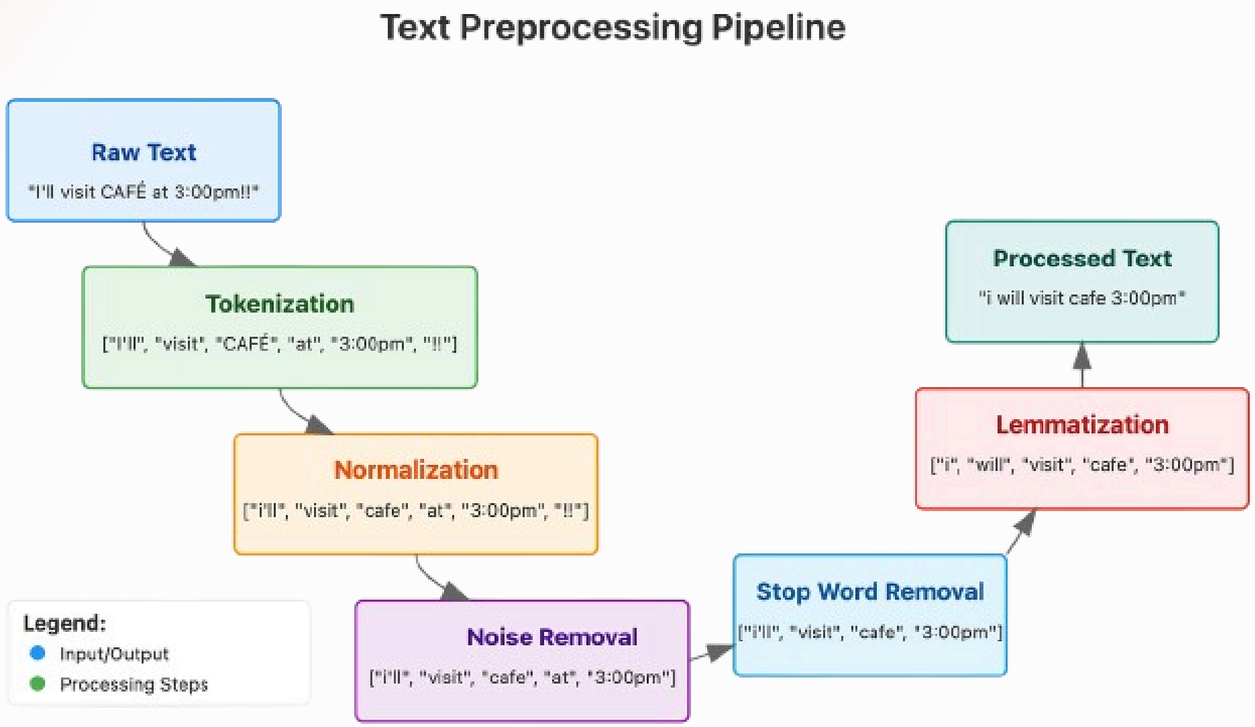
NLP

Natural Language Processing (NLP) is a field of Artificial Intelligence (AI) that focuses on enabling computers to understand, interpret, and generate human language (text or speech).



NLP is essential to Judge It Well as it converts complex, unstructured legal text into structured data, enabling accurate legal reasoning, document analysis, and task-specific predictions by the fine-tuned LLM.

Preprocessing



- Tokenization: Breaking text into units (words/sub-words).
- Normalization: Case-folding, removing Stop Words ("the", "is").
- Stemming vs. Lemmatization: Chopping words to roots (Stemming) vs. finding dictionary roots (Lemmatization).
- Advanced Tagging: Identifying Parts of Speech (POS) and Named Entities (NER).

Text Representation

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

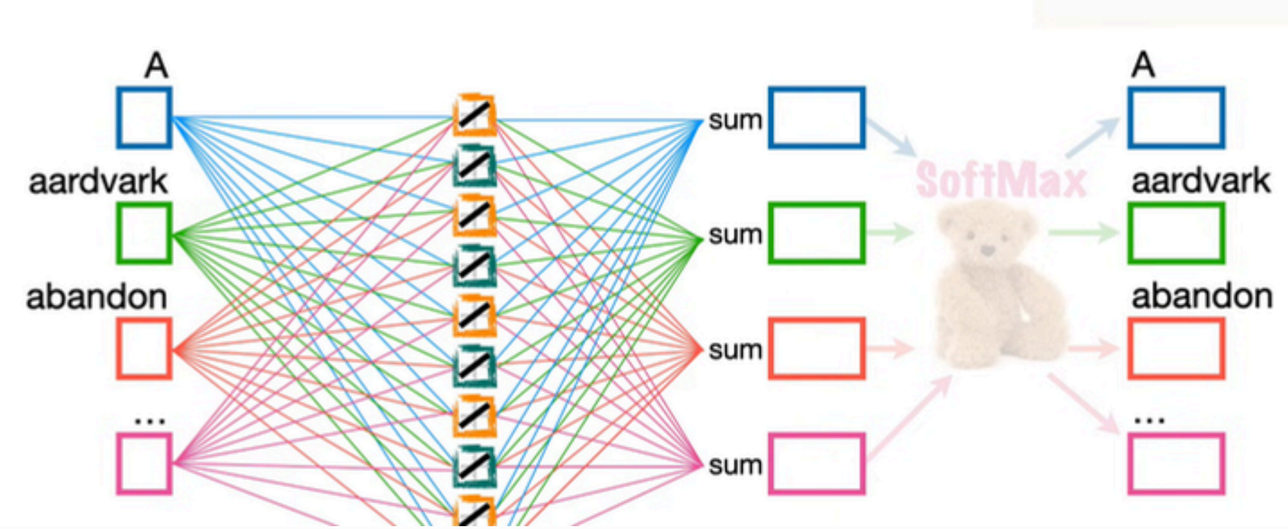
$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

$$IDF = log(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term} + 1}$$

$$TF\text{-}IDF = TF * IDF$$

- Bag of Words (BoW): Counts word frequency without considering order or context.
- One-Hot Encoding: Represents words as binary vectors, leading to high memory usage.
- TF-IDF: Assigns higher importance to rare, meaningful words over common ones.

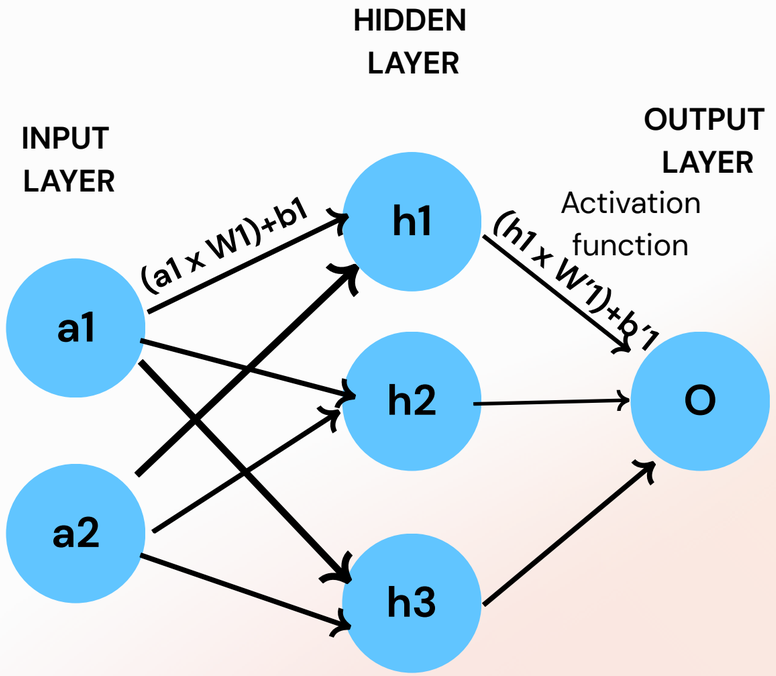
Word Embedding



- Word2Vec: Learns context. CBOW predicts word from context
- GloVe: Captures global corpus statistics, not just local neighbors.
- FastText: Breaks words into sub-parts (n-grams); handles unseen words well.
- BERT: Transformer-based; reads bidirectionally for deep, contextual understanding.

ANN

Artificial Neural Networks (ANNs) are computational models composed of interconnected layers of neurons that learn mappings from inputs to outputs by adjusting weights and biases.



Each neuron computes a weighted sum of inputs followed by a non-linear activation function, enabling the network to model complex, non-linear relationships.

ANNs are trained using backpropagation and gradient descent and are widely used in tasks such as classification, regression, and pattern recognition.

Backpropagation

Backpropagation trains neural networks by computing the gradient of the loss with respect to each weight and updating them using gradient descent.

Forward pass → Loss Function → Backward pass(chain rule) → Weight update

$$z = Wx + b \quad a = \sigma(z)$$

$$\mathcal{L}(y, \hat{y})$$

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial W}$$

$$W = W - \eta \frac{\partial \mathcal{L}}{\partial W}$$

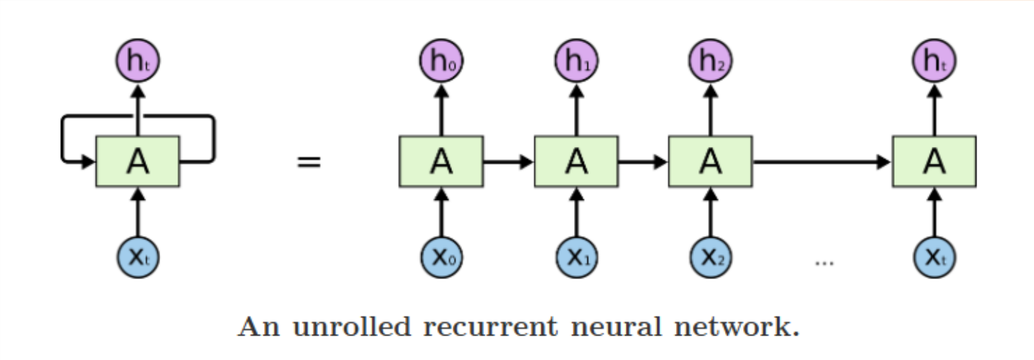
Backpropagation tells each weight how much to change to reduce the overall error.

RNN

Recurrent Neural Networks (RNNs) model sequential data by reusing weights over time and maintaining a hidden state to capture temporal dependencies.

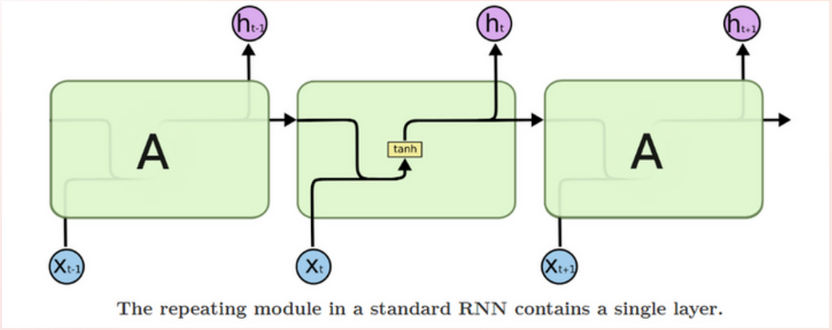
The hidden state combines the current input with past context.

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$
$$y_t = g(W_y h_t)$$



An unrolled recurrent neural network.

- Natural language is sequential and order-dependent
- RNNs capture context and word dependencies



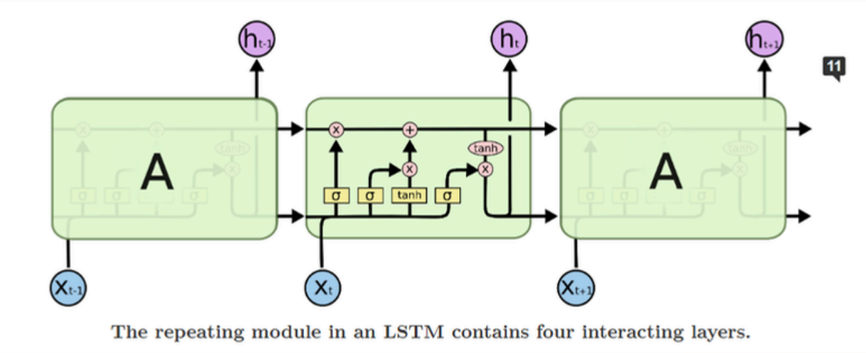
The repeating module in a standard RNN contains a single layer.

RNNs extend neural networks with memory, making them suitable for sequence-based tasks like natural language processing.

LSTM

Traditional RNNs struggle to learn long-term dependencies in sequential data due to the vanishing and exploding gradient problem.

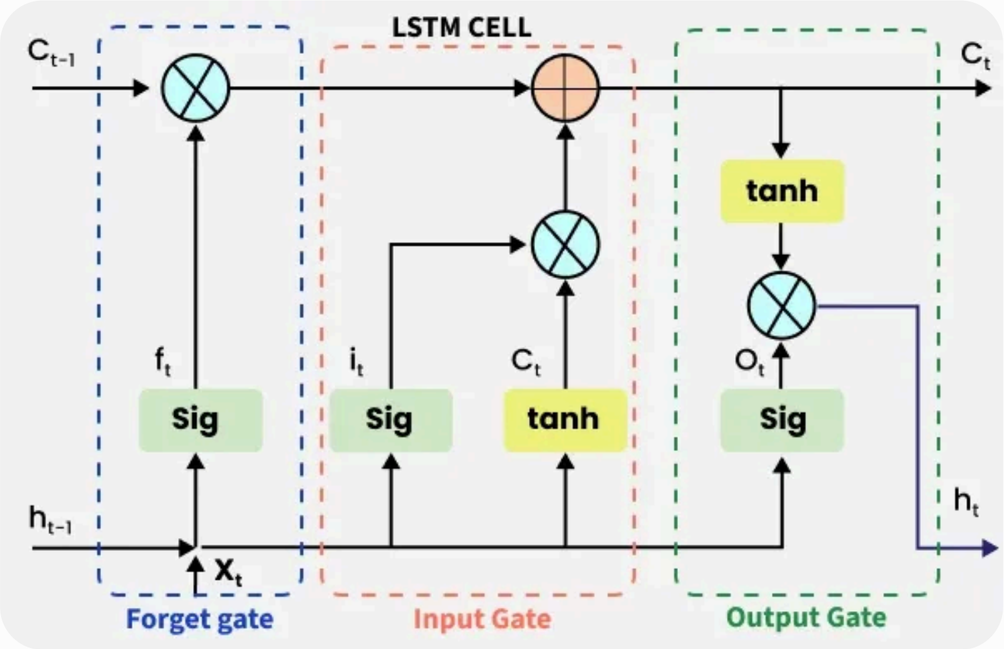
In NLP, vanishing and exploding gradients prevent RNNs from learning long-range dependencies, such as linking words across long sentences and preserving semantic context.



The repeating module in an LSTM contains four interacting layers.

LSTMs solve this problem using gated memory cells that control information flow, allowing important linguistic context to be retained over long sequences.

LSTM Architecture



LSTM architecture consists of 3 gates:

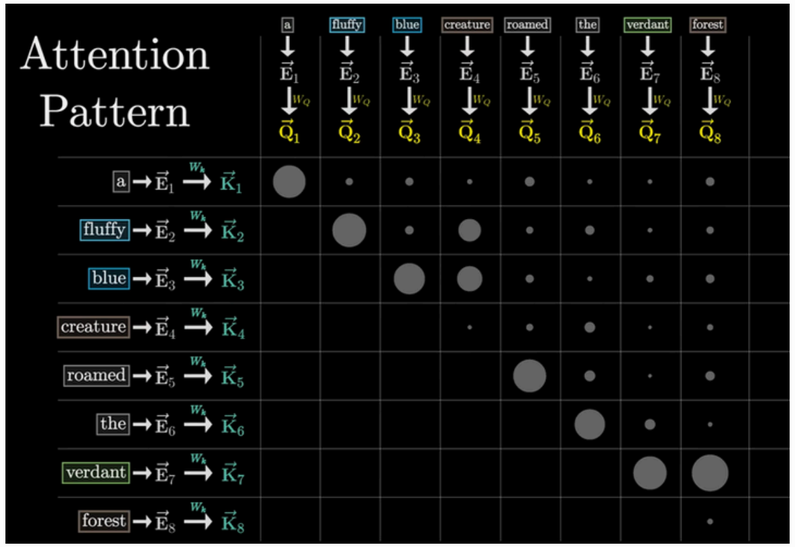
FORGET GATE: The information that is no longer useful in the cell state is removed with the forget gate.

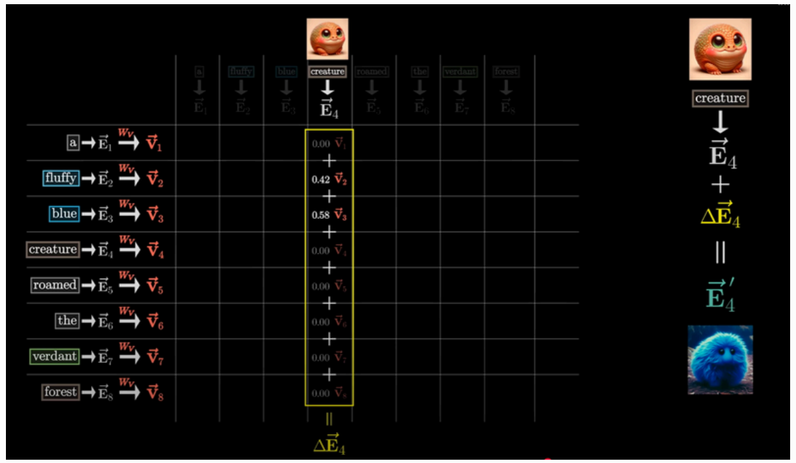
INPUT GATE: The addition of useful information to the cell state is done by the input gate.

OUTPUT GATE: The output gate is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step.

ATTENTION!

- Parallel Processing
- Long-Range Dependencies
- Scalability

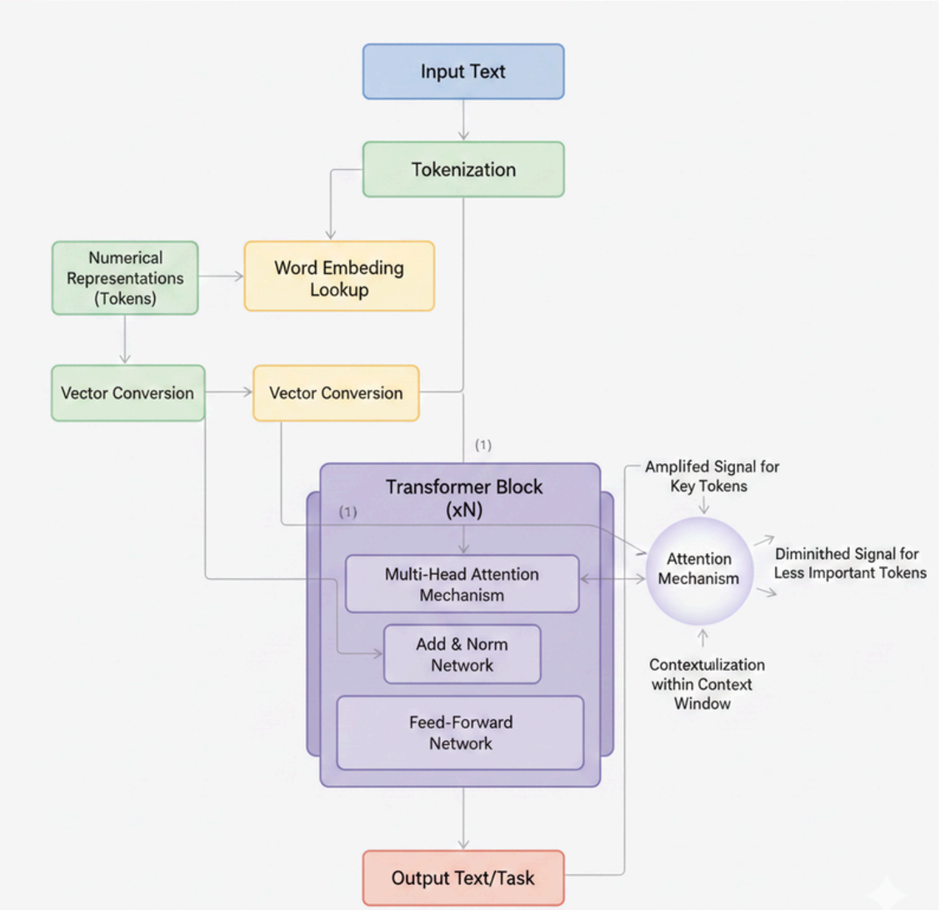


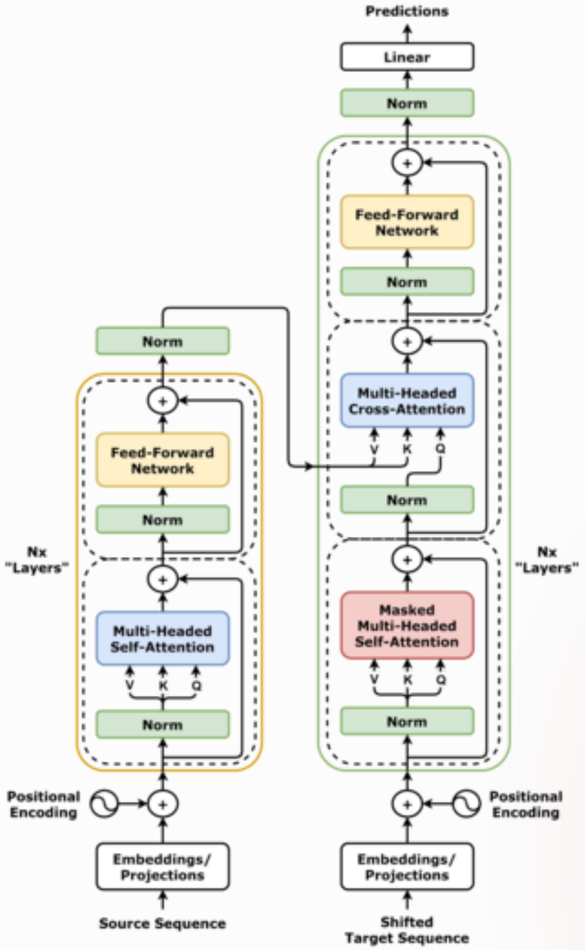


- Allows models to focus on specific parts of the input sequence - It assigns different weights to different input elements enabling the model to prioritize certain information over others.
- **Scaled Dot-Product Attention:** queries (Q), keys (K) and values (V)
- **Multi-Head Attention:** It involves multiple attention heads each with its own set of query, key and value matrices. The outputs of these heads are concatenated and linearly transformed to produce the final output.
- **Self-Attention:** enables the model to capture long-range dependencies and relationships within the input sequence.

Transformers

WHAT DO THEY DO?





No recurrent units unlike RNNs

The encoder–decoder architecture:
The encode - encoding layers that process all the input tokens together one layer after another.

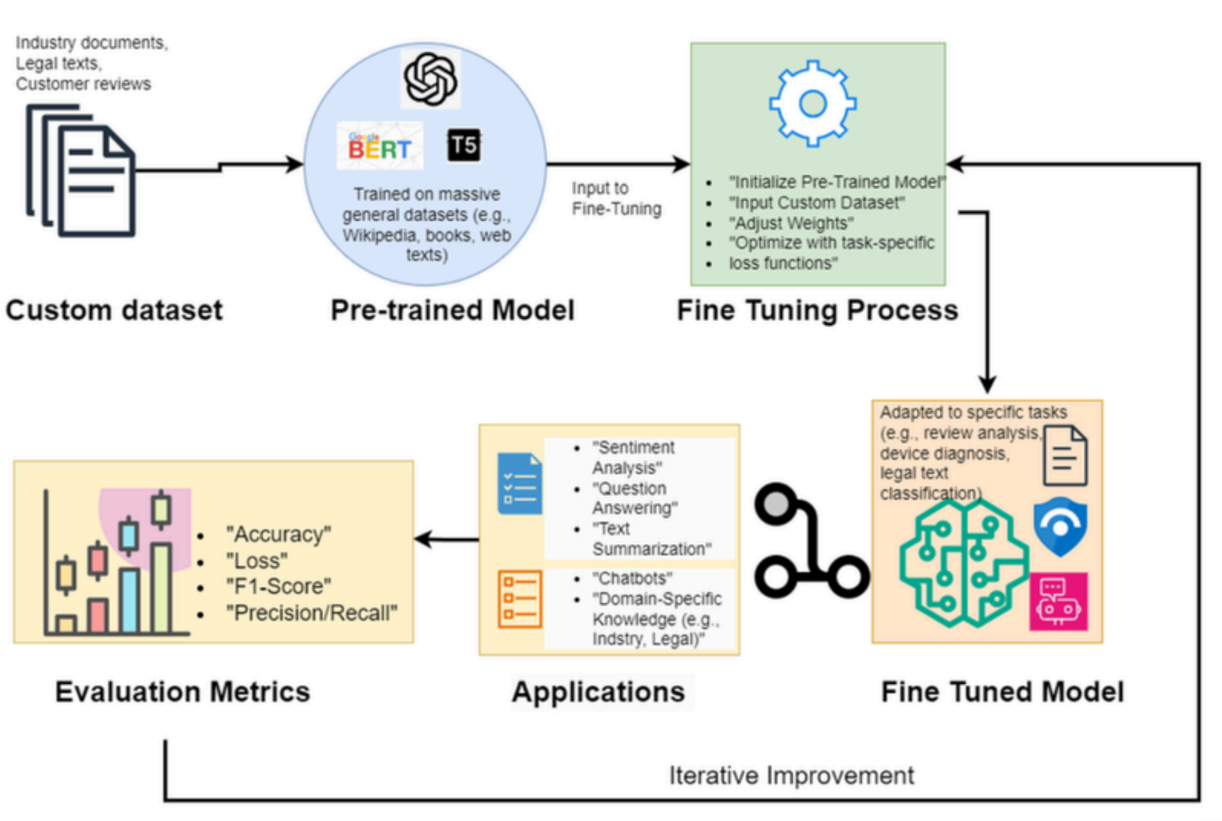
The decoder decoding layers that iteratively process the encoder's output and the decoder's output tokens so far.

Feed-forward neural network - additional processing of their outputs and contain residual connections and layer normalization steps. They contain most of the parameters in a transformer model.

PRETRAINED LLM

- High-Efficiency Architecture: Llama 3 base model optimized via Unsloth for memory-efficient training on standard hardware.
- Specialized Legal Fine-Tuning: Rigorous ingestion of IPC, CrPC, and Constitution data to bridge the gap from general AI to legal expert.
- Law-Aware Engine Output: A domain-specific system providing precise, formal, and contextually accurate answers to complex legal inquiries.

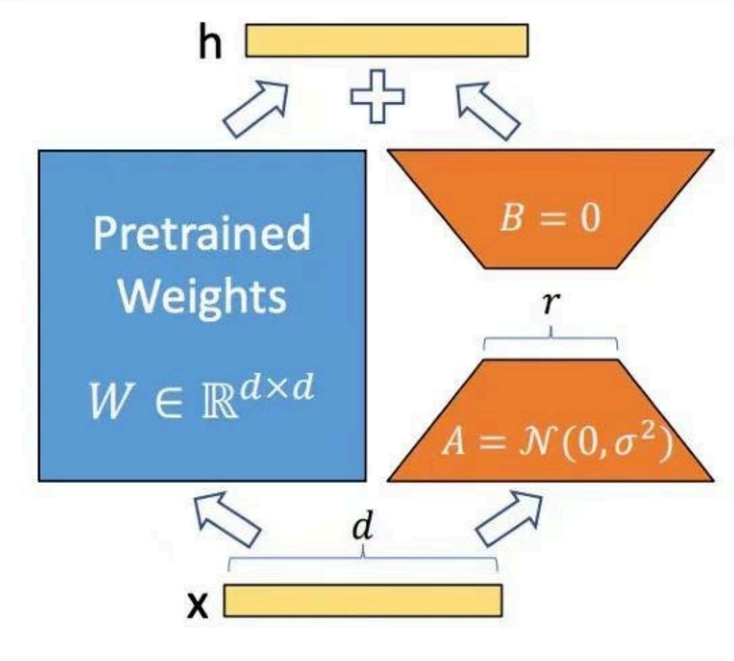
FINE TUNING



Fine-tuning of a Large Language Model (LLM) is the process of adapting a pre-trained model to perform well on a specific task or domain.

Fine-tuning is efficient because it requires far less data and computational resources than training a model from scratch. It also improves accuracy, consistency, and usefulness in real-world scenarios.

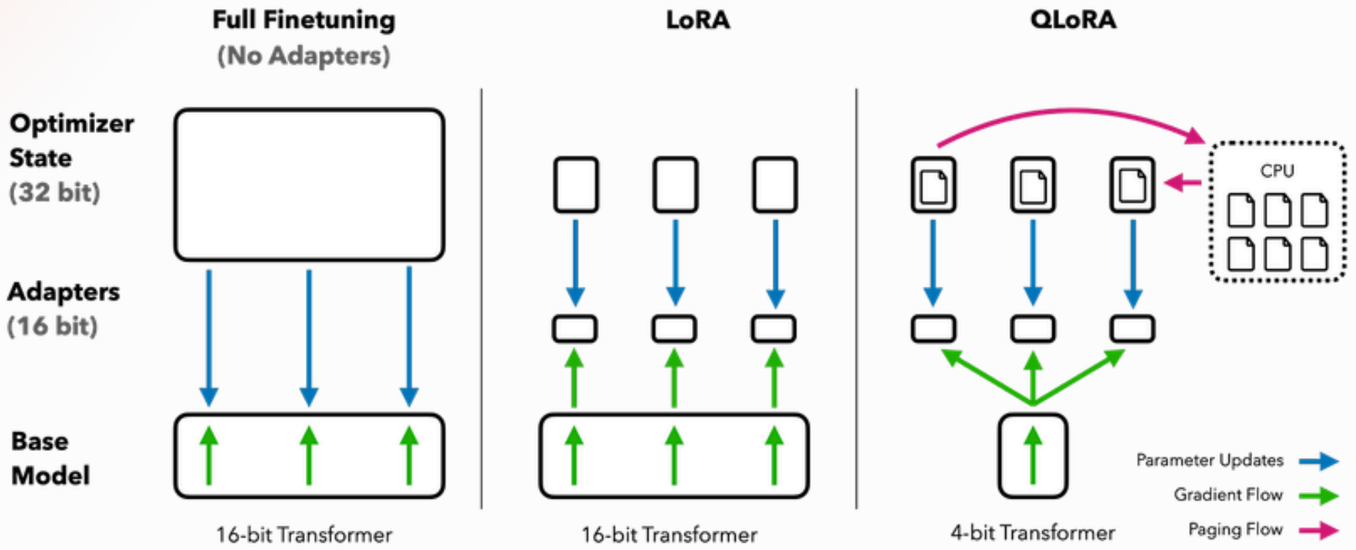
LORA MODEL



LORA stands for Low Rank Adaptation. It is used for fine-tuning without changing the original weights. It also saves a lot of memory as it uses less parameters. LoRA model does not change original weights but updates them.

Updated Weights **W' = W + ΔW**, where **ΔW= A.B**
Where A and B have much lower rank than W.

QUANTIZATION



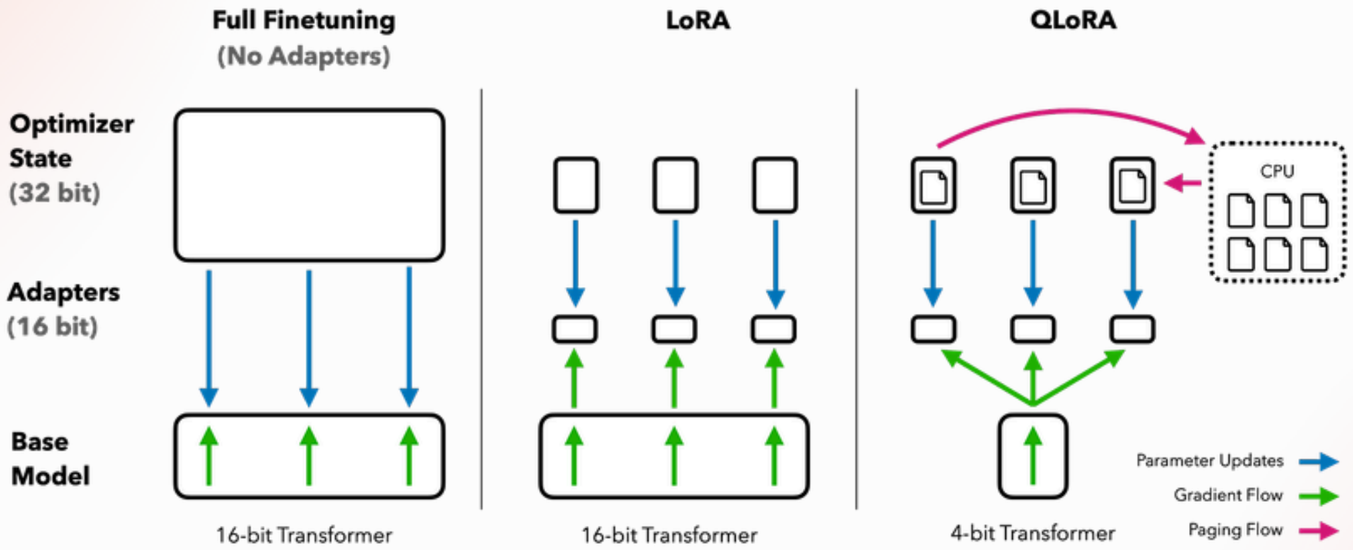
To make the training speed much faster methods like FP16, FP8, or INT8 quantization are employed. The conversion is done through scaling:-

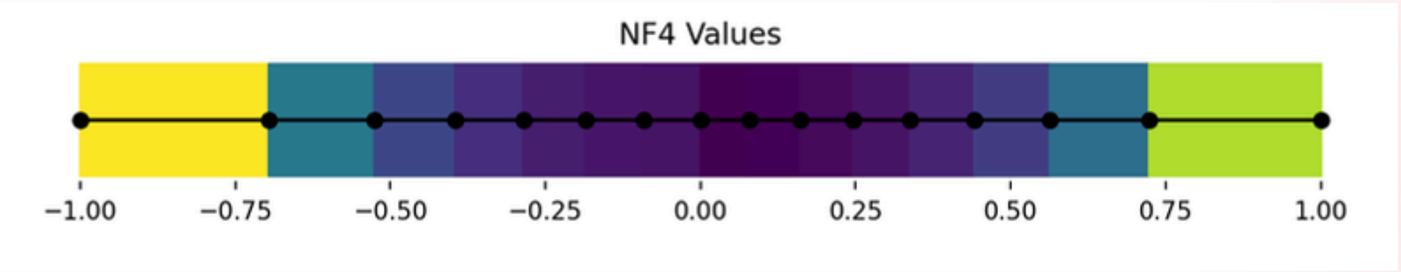
$$scale = \frac{x_{max} - x_{min}}{q_{max} - q_{min}}$$

where x is unit converted to and q is converted from.

But QLORA makes it even faster!!

QLORA MODEL

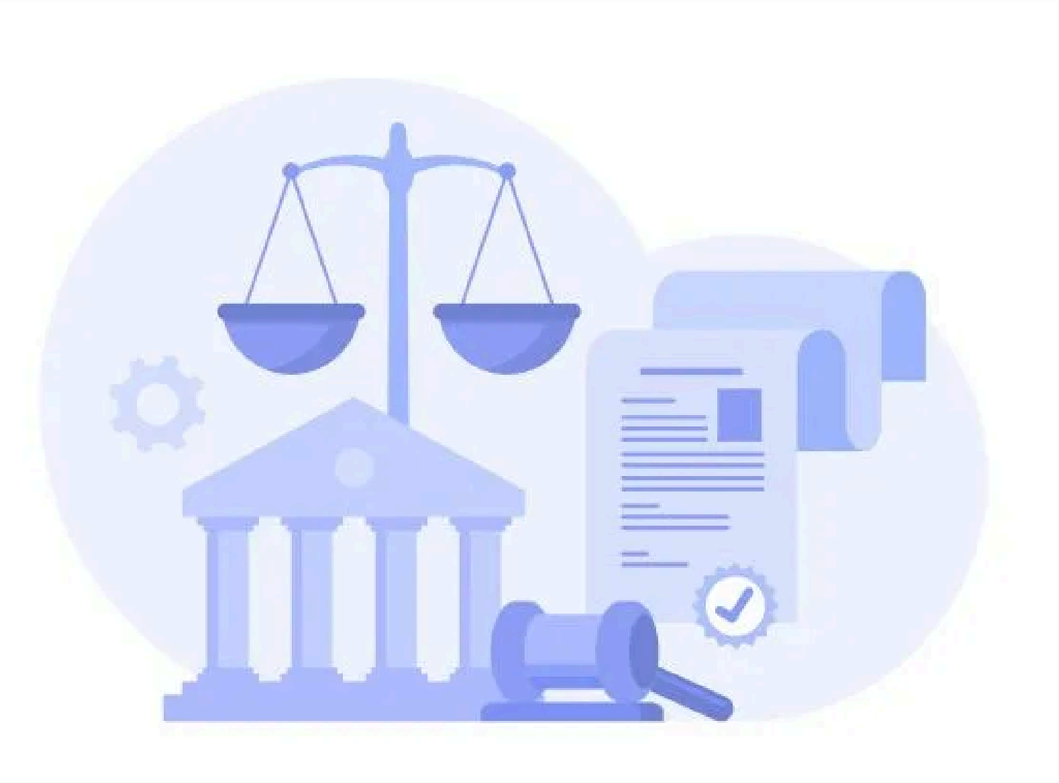




QLORA is quantized form of LORA model. QLoRA uses Normalized Float 4-bit (NF4) quantization. NF4 maintains precision by normalizing values in a way that aligns well with deep neural networks.

NF4 :- In this quantization regular scaling is not done as most weights are closed to 0. Scaling is done such that each int4 value is mapped by equal percentages of weights. Hence distribution is non-uniform.

Legal Dataset



- Comprehensive Corpus: Aggregated IPC, CrPC, and the Indian Constitution, integrated with 33,000+ judicial judgments on crimes and sentencing.
- Knowledge Evolution: Shifts the model from General English comprehension to Specialized Indian Lawreasoning and terminology.
- Granular Q&A Format: Fine-tuned on precise instruction pairs (Question/Answer) to ensure exactness in procedural details and statutory amendments.

ABOUT UNSLOTH

Unsloth is a Python library designed to make fine-tuning large language models (LLMs) faster, cheaper, and more memory-efficient.

ADVANTAGES:-

- 2×–5× faster training
- Single GPU required for 7B parameters
- Wraps around Hugging Face Transformers
- Uses LoRA / QLoRA

IMPLEMENTATION

THE FINAL CODE

OF THE PROJECT

Using Pytorch and Unsloth

```
# Import torch to check GPU details
import torch

# Get the CUDA compute capability of the current GPU
major_version, minor_version = torch.cuda.get_device_capability()

# Install Unsloth from GitHub (patched for latest Colab)
!pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"

# Install required libraries without touching Colab's PyTorch
!pip install --no-deps "xformers<0.0.27" "trl<0.9.0" peft accelerate bitsandbytes

# Fix UTF-8 encoding issue that sometimes breaks progress bars
import locale
locale.getpreferredencoding = lambda: "UTF-8"
```

Loading Llama 3.1 8B model from unsloth

```
# 1. Load the model
max_seq_length = 2048
dtype = None  # Auto-detect
load_in_4bit = True  # Use 4bit quantization to reduce memory
```

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Meta-Llama-3.1-8B-bnb-4bit",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
)
```

Loading LoRA Model

```
# 2. Add LoRA adapters
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,  # LoRA rank
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 3407,
)
model.print_trainable_parameters()
```

```
trainable params: 41,943,040
all params: 8,072,204,288
```

Huge reduction in training parameters!!

```
trainable%: 0.5196
```

Formatting the dataset such that the model can understand

```
system_prompt = example['instruction'] # Assuming instruction is part of the example
user_input = example['input']
model_output = example['output']

formatted_text = (
    f"<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n"
    f"{system_prompt}<|eot_id|><|start_header_id|>user<|end_header_id|>\n"
    f"{user_input}<|eot_id|><|start_header_id|>assistant<|end_header_id|>\n"
    f"{model_output}<|eot_id|>"
)
```

Instructing the model by giving prompt

```
# Llama-3 chat prompt template
legal_prompt = """<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are an expert Indian Legal Assistant. Answer strictly based on the provided context.<|eot_id|><|start_header_id|>user<|

Question: {}<|eot_id|><|start_header_id|>assistant<|end_header_id|>

{}"""

# End-of-sequence token
EOS_TOKEN = tokenizer.eos_token
```

```
def formatting_prompts_func(examples):
    questions = examples["question"]
    answers = examples["answer"]
    formatted_texts = []

    for question, answer in zip(questions, answers):
        prompt = legal_prompt.format(question, answer)
        formatted_texts.append(prompt + EOS_TOKEN)

    return {"text": formatted_texts}

# Apply formatting to dataset
dataset = dataset.map(formatting_prompts_func, batched=True)

# Sanity check
print("Data formatted successfully")
print(dataset[0]["text"])
```

Training the model on dataset

```
rom trl import SFTTrainer
rom transformers import TrainingArguments
mport torch

 Setup SFT trainer
rainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    packing=False,
    args=TrainingArguments(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        warmup_steps=5,
        max_steps=60,
        learning_rate=2e-4,
        fp16=not torch.cuda.is_bf16_supported(),
        bf16=torch.cuda.is_bf16_supported(),
        logging_steps=1,
        optim="adamw_8bit",
        weight_decay=0.01,
        lr_scheduler_type="linear",
        seed=3407,
        output_dir="outputs",
    ),
```

```
# GPU memory info before training
gpu = torch.cuda.get_device_properties(0)
reserved = round(torch.cuda.max_memory_reserved() / 1024**3, 3)
total = round(gpu.total_memory / 1024**3, 3)

print(f"GPU: {gpu.name} | Total Memory: {total} GB")
print(f"Reserved Memory: {reserved} GB")

print("Starting training...")
trainer_stats = trainer.train()
print("Training finished.")
```

Output

test_question = "bad comments on women punishments ?"
Legal AI Answer: The punishment for a person who makes or publishes any imputation on the character of a deceased woman is imprisonment for a term which may extend to two years, or with fine, or with both.

```python
FastLanguageModel.for_inference(model) # Enable fast inference

# Define a test question
test_source = "IPC"
test_question = "bad comments on women punishments ?"

# Prepare the input
input_text = legal_prompt.format(test_source, test_question, "") # Leave answer blank

inputs = tokenizer([input_text], return_tensors = "pt").to("cuda")

# Generate response
outputs = model.generate(**inputs, max_new_tokens = 128, use_cache = True)
response = tokenizer.batch_decode(outputs)[0]
  ⌘I for Command, ⌘L for Agent
# Extract and print only the assistant's answer
print("Legal AI Answer:")
print(response.split("<|start_header_id|>assistant<|end_header_id|>")[-1].replace(EOS_TOKEN, ""))
```

Python

```
Legal AI Answer:


The punishment for a person who makes or publishes any imputation on the character of a deceased woman is imprisonment for a term which may extend
```

Final Q&A comparison:

test_question: what is the punishment of murder in india?

Fine tuned llama

In India, the punishment for murder is death penalty or life imprisonment, along with a fine, under Section 302 of the Indian Penal Code.

Usual llama

Punishment of murder in India is death or life imprisonment and also fine.

As observed, the fine-tuned LLaMA model produces more accurate and context-aware responses compared to the base LLaMA model, which tends to generate generic answers and lacks specific legal terminology and statutory references.

```
prompt = "Question: What is the punishment of murder in India?\nAnswer:"

inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

outputs = model.generate(
    **inputs,
    max_new_tokens=150,
    temperature=0.7,
    top_p=0.9,
    do_sample=True
)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

```
Question: What is the punishment of murder in India?
Answer: Punishment of murder in India is death or life imprisonment and also fine.
```

GPT 2 outcome fine tuned

Test question: 'What is the title and extent of operation of the Indian Penal Code?

Answer: "The title is 'The Indian Penal Code' and its operation extends to the punishment of offences committed within India, and beyond but which by law may be tried within India. It also includes extension of the Code to extra-territorial offences."

Usual GPT 2 outcome

Test question: 'What is the title and extent of operation of the Indian Penal Code?

Answer:The Indian Penal Code has been amended in accordance with the Indian Penal Code as amended by the Bill of Rights Act. The Bill of Rights Act provides for the judicial and administrative powers of the Indian Government to issue decrees and orders for the purposes of the Indian Penal Code. The Indian Penal Code provides that the Indian Penal Code shall be amended to provide for the judicial and administrative powers of the Indian Government

HOW IS OUR MODEL DIFFERENT

Fine-Tuned Indian Legal LLM

- Plus
- High accuracy for Indian law
- Uses sections & case laws
- Low cost, on deployment
- Fast, exam & practice oriented

vs

ChatGPT-like Large LLM

- Drawback of chatgpt like llm
- Expensive & cloud-dependent
- Less jurisdiction-specific
- Often vague for legal precision
- Avoids strong legal opinions

CONCLUSION

- End-to-End Mastery: Built the entire stack from scratch—from raw text cleaning to training Neural Networks—understanding the "why," not just the "how."
- Democratized AI: Proved that LoRA & Quantization allow training massive models on standard laptops, bypassing the need for supercomputers.
- Domain Expertise: Transformed a "General" AI into a secure, private Indian Legal Specialist, solving real-world problems without cloud dependency.

Thank You